Algorytm Dijkstry zrównoleglony metodą delta-stepping

Karol Helm, Maciej Anioł

Listopad 2026

1 Wprowadzenie

Celem projektu było zimplementowanie oraz przebadanie uwspółbieżnionej wersji algorytmu Dijkstry wykorzystującej metodę delta-stepping opisaną w artykule U. Meyer and P. Sanders, Δ-stepping: a parallelizable shortest path algorithm, Journal of Algorithms 49 (2003).
Dany jest graf skierowany o nieujemnych wagach całkowitych. Obliczamy długości najkrótszych ścieżek z wierzchołka 0 do wszystkich wierzchołków. Zwracamy odległość do wierzchołka $n-1$ lub -1, jeśli wierzchołek $n-1$ jest nieosiągalny.
Paczka projektu zawiera:

- Rozwiązanie jednowątkowe *dijkstraOneThread.cpp*
- Rozwiązanie wielowątkowe *dijkstraDeltaStepping.cpp*
- Raport *raport.pdf*
- Skrypt *run.sh* uruchamiający *scaling.py*
- Program w Pythonie generujący grafy oraz porównujący czasy działania rozwiązań *scaling.py*
- Prezentację na zajęcia *prezentacja.pdf*

2 Opis wersji jednowątkowej i wielowątkowej

2.1 Wersja jednowątkowa

Wersja jednowątkowa jest implementacją klasycznego algorytmu Dijkstry korzystającego z kolejki priorytetowej.

1. Zdejmujemy z kolejki wierzchołek o najmniejszej odległości.
2. Sprawdzamy, czy ta odległość jest aktualna.
3. Relaksujemy krawędzie i jeśli poprawiamy odległość do innego wierzchołka to wrzucamy go na kolejkę.

Konieczność utrzymania konkretnej kolejności przetwarzania wierzchołków utrudnia efektywne zrównoleglenie tej wersji algorytmu.

2.2 Wersja wielowątkowa

Wersja wielowątkowa korzysta z techniki delta-stepping. Zamiast wybierania pojedynczego wierzchołka o minimalnej odległości, algorytm przetwarza wierzchołki według kubełków odpowiadających przedziałom odległości $[i\Delta, (i+1)\Delta)$. Współczynnik Δ może być dowolną dodatnią liczbą, ale jego wybór ma wpływ na czas działania algorytmu. W naszej implementacji to $max(1, floor(2W/m))$, gdzie W to suma wag, a m to liczba krawędzi. Dla $m = 0$ przyjmujemy $\Delta = 1$. Przed rozpoczęciem właściwego algorytmu dla każdego wierzchołka dzielimy krawędzie na lekkie ($waga <= \Delta$) oraz ciężkie ($waga > \Delta$). Lekkie krawędzie mogą poprawiać odległości wewnątrz aktualnego kubełka, natomiast ciężkie na pewno prowadzą do późniejszych kubełków od aktualnie przetwarzanego. Pozwala to odroczyć ciężkie relaksacje do zakończenia fazy lekkiej i ograniczyć liczbę operacji. Zaczynamy od jednego kubełka zawierającego wierzchołek 0 z odległością 0. Następnie dopóki wszystkie kubełki nie są puste:

1. Wybieramy najmniejszy niepusty kubełek.
2. Relaksujemy równolegle krawędzie lekkie zapisując ich wynikowe pozycje do bufora
3. Jeden wątek przenosi pozycje z buforów do kubełków. Jeśli aktualny kubełek nie jest pusty, to powtarzamy proces relaksacji lekkich krawędzi
4. Po zakończeniu fazy lekkiej relaksujemy równolegle krawędzie ciężkie. Korzystamy z tablicy wartości *added*, aby każdy wierzchołek trafił do fazy ciężkiej tylko raz
5. Ponownie jeden wątek przenosi wierzchołki z buforów do kubełków

Wersja wielowątkowa korzysta z biblioteki OpenMP. Przetwarzanie kubełków odbywa się w bloku `omp parallel`. Pętla, która przechodzi przez wierzchołki obecnego kubełka, jest podzielona przez `omp for`. Bufor przy relaksacji jest konieczny, ponieważ operacja `push_back` nie jest atomowa. Przy scalaniu buforów korzystamy z `omp single`. Odległości przechowujemy w wektorze elementów atomowych. Funkcja `compare_exchange_weak` pozwala bezpiecznie zaktualizować odległość, kiedy kilka wątków jednocześnie znajduje różne drogi do tego samego wierzchołka. Sekwencyjność pozostaje przy wczytaniu i inicjalizacji danych, budowie grafu, podziale na krawędzie light i heavy oraz wcześniej wspomnianym scalaniu buforów.

3 Pomiar wydajności algorytmu

Wydajność była badana zarówno na urządzeniu lokalnym (CPU: Intel Core i7-1260P), jak i na serwerze (CPU: 4 x Xeon Gold 6154). Wyniki prezentują się następująco:

Algorytm	Wątki	Środowisko	Random 1 [s]	Random 2 [s]	Random 3 [s]	Path [s]	Warstwowy [s]
Dijkstra	1	Serwer	4.9897	4.9347	4.5255	**≈ 0.0001**	1.7154
Dijkstra	1	Lokalnie	4.9134	4.8148	6.7988	**≈ 0.0004**	2.3736
Delta-stepping	1	Serwer	2.3455	2.5360	2.3838	0.0369	0.9053
Delta-stepping	1	Lokalnie	3.1423	4.0485	**2.3930**	0.0185	1.2473
Delta-stepping	2	Serwer	2.0267	2.1500	2.0804	0.0517	0.8797
Delta-stepping	2	Lokalnie	4.0094	3.2762	4.3620	0.0361	1.0048
Delta-stepping	4	Serwer	1.6348	1.7260	1.7009	0.0563	0.8052
Delta-stepping	4	Lokalnie	3.4330	**1.7054**	3.8453	0.0465	**0.9176**
Delta-stepping	8	Serwer	1.4132	**1.5012**	1.4500	0.1110	**0.7710**
Delta-stepping	8	Lokalnie	3.2629	3.6966	3.3817	0.0991	1.2976
Delta-stepping	16	Serwer	**1.3812**	1.5050	**1.3683**	0.2955	0.7807
Delta-stepping	16	Lokalnie	**3.0442**	3.8418	4.2198	8.3398	2.5401
Delta-stepping	72	Serwer	5.8073	7.9729	3.6094	—	—
Delta-stepping	72	Lokalnie	3.1629	2.9860	2.4017	—	—

Rysunek 1: Mediany czasu działania algorytmów na serwerze i lokalnie, bez operacji wejścia oraz wyjścia. Czasy podano w sekundach. Tabelę przygotowano w programie Excel.

Rozwiązania zostały przebadane na:

1. Trzech grafach Random - losowe grafy na 4 mln wierzchołków i na 16 mln krawędzi. Rozwiązania zostały uruchomione na każdym grafie z tej grupy po trzy razy, po czym obliczono medianę.

2. Grafie Path - ścieżka na 10 tys. wierzchołków, zostało wykonanych po 5 pomiarów i wybrano medianę.

3. Grafie warstwowym na 2 mln wierzchołków. Wykonano 5 pomiarów i wpisano do tabeli medianę.

Analiza uzyskanych danych znajduje się w sekcji Wnioski.

4 Kompilacja i uruchomienie

Domyślne uruchomienie programu następuje poprzez wywołanie skryptu *run.sh*, który uruchamia *scaling.py* - program w Pythonie odpowiedzialny za skompilowanie rozwiązań oraz wygenerowanie grafów testowych, na których następnie czasy rozwiązań są porównywane. Oczywiście można też skompilować i przetestować rozwiązania ręcznie komendami:

```
g++ src/dijkstraOneThread.cpp -O3 -std=c++17 -o dijkstraOneThread
g++ src/dijkstraDeltaStepping.cpp -O3 -std=c++17 -fopenmp -o dijkstraDeltaStepping
```

Oba programy przyjmują wejście w postaci kolejnych linii:

1. liczba zestawów danych z

2. liczba wierzchołków n i krawędzi m

3. m linii opisujących krawędzie skierowane w postaci u, v, val.

5 Wnioski

Nasza implementacja zrównoleglonego algorytmu Dijkstry przy uruchomieniu na serwerze uzyskała przyspieszenie 3,61×, 3,29×, 3,31× na grafach losowych oraz 2,22× na grafie warstwowym. Przy uruchomieniu lokalnym uzyskano przyspieszenie 1,61×, 2,82×, 2,84× na grafach losowych oraz 2,59× na grafie warstwowym. Mnożnik został wybrany jako najbardziej efektywne dobranie liczby wątków przy każdym grafie. Niestety delta-stepping okazuje się nie być efektywny dla ścieżki. Straty wynikikające z wielowątkowości znacznie przewyższają zyski, przez co program dla ścieżki działa wolniej.